Laboratoire Méthodes Formelles

CentraleSupélec | université PARIS-SACLAY

DÉVELOPPEMENT DE SYSTÈMES CRITIQUES AVEC LA MÉTHODE EVENT-B

LE PLUGIN THEORY DE RODIN/EVENT-B

3A cursus ingénieurs - Mention Sciences du Logiciel
CentraleSupelec - Université Paris-Saclay - 2025/2026

Idir AIT SADOUNE
idir.aitsadoune@centralesupelec.fr

THE RODIN PLATFORM

Required plugins for this tutorial :

menu : `Help -> Install New Software ...`

- the **Atelier B Provers plugin** from the **Atelier B Provers** Update site.
  `https://www.atelierb.eu/update_site/atelierb_provers`
- the **ProB plugin** from the **ProB** Update site.
  `https://stups.hhu-hosting.de/rodin/prob1/release/`
- the **Theory plugin** from the **Rodin Plug-ins (archive)** Update site.
  `https://rodin-b-sharp.sourceforge.net/updates-archive`

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

THE EVENT-B METHOD

```
CONTEXT ctx1
EXTENDS ctx2

SETS s
CONSTANTS c
AXIOMS
  A(s, c)
THEOREMS
  T(s, c)
END
```

```
MACHINE mch1
REFINES mch2
SEES ctxi

VARIABLES v
INVARIANTS
  I(s, c, v)
THEOREMS
  T(s, c, v)
EVENTS
  [events_list]
END
```

```
event ≙
  any x
  where
    G(s, c, v, x)
  then
    BA(s, c, v, x, v')
  end
```

$$A(s,c) \quad \vdash \quad T(s,c)$$
$$A(s,c) \wedge I(s,c,v) \quad \vdash \quad T(s,c,v)$$
$$A(s,c) \wedge I(s,c,v) \wedge G(s,c,v,x) \quad \vdash \quad \exists v'.\, BA(s,c,v,x,v')$$
$$A(s,c) \wedge I(s,c,v) \wedge G(s,c,v,x) \wedge BA(s,c,v,x,v') \quad \vdash \quad I(s,c,v')$$
$$\ldots$$

CentraleSupélec
université PARIS-SACLAY

LIMITES DU LANGAGE MATHÉMATIQUE NATIF

- Types prédéfinis ($\mathbb{Z}$, $BOOL$, ensembles)
- Opérateurs standards
- Difficulté à modéliser :
  - ➠ des structures de données spécifiques
  - ➠ des domaines métiers complexes
  - ➠ des calculs spécialisés (temps, graphes, files, etc.)

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

MOTIVATION

- Réutilisabilité des concepts mathématiques
- Modularité et abstraction
- Extension contrôlée du langage **Event-B**

CentraleSupélec
université PARIS-SACLAY

QU'EST-CE QU'UNE THÉORIE ?

Un composant définissant :

- des types abstraits
- des opérateurs
- des axiomes
- des théorèmes
- des règles de preuve

Positionnement dans **Rodin**

- Intégration au niveau des contextes/machine
- Impact sur la génération des obligations de preuve
- Interaction avec les prouveurs automatiques et interactifs

CentraleSupélec
université PARIS-SACLAY

THE THEORY PLUGIN

- **Theory Plug-in** provides capabilities to extend **the Event-B mathematical language** and **the Rodin proving infrastructure**.

- An **Event-B theory** can contain :
  - new datatype definitions,
  - new polymorphic operator definitions,
  - axiomatic definitions,
  - theorems,
  - associated rewrite and inference rules.

- Michael J. Butler and Issam Maamria.
  Practical theory extension in Event-B. Theories of Programming and Formal Methods. 2013.
- Thai Son Hoang, Laurent Voisin, Asieh Salehi, Michael J. Butler, Toby Wilkinson, and Nicolas Beauger.
  Theory plug-in for Rodin 3.x. CoRR, abs/1701.08625, 2017.

CentraleSupélec
université PARIS-SACLAY

THE EVENT-B METHOD

```
THEORY thy_1
IMPORT thy_2

TYPE PARAMETERS
  TP_1, ..., TP_n
DATATYPES
  DT_1, ..., DT_n
OPERATORS
  OP_11, ..., OP_1n
AXIOMATIC DEFINITIONS
  operators
    OP_21, ..., OP_2n
  axioms
    A
THEOREMS
  T
PROOF RULES
  PR
END
```

```
CONTEXT ctx_1
EXTENDS ctx_2

SETS s
CONSTANTS c
AXIOMS
  A(s, c)
THEOREMS
  T(s, c)
END
```

```
MACHINE mch_1
REFINES mch_2
SEES ctx_i

VARIABLES v
INVARIANTS
  I(s, c, v)
THEOREMS
  T(s, c, v)
EVENTS
  [events_list]
END
```

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

TYPES ABSTRAITS (ABSTRACT TYPES)

Un **type abstrait** est un type :

- sans implémentation concrète
- aucune hypothèse sur sa représentation interne
- défini uniquement par ses propriétés (axiomes)
- manipulé via des opérateurs

➠ On raisonne sur ce que c'est, pas sur comment c'est implémenté.

```
THEORY REAL
AXIOMATIC DEFINITIONS
  TYPES
    ℝ
  OPERATORS
    zero : ℝ
    one : ℝ
    plus(a ∈ ℝ, b ∈ ℝ) : ℝ INFIX
    sub(a ∈ ℝ, b ∈ ℝ) : ℝ INFIX
    mult(a ∈ ℝ, b ∈ ℝ) : ℝ INFIX
    div(a ∈ ℝ, b ∈ ℝ) : ℝ INFIX
      well-definedness condition: b ≠ zero
    ...
    lt(a ∈ ℝ, b ∈ ℝ) : BOOL INFIX
    ...
  AXIOMS
    ∀ x,y · (x plus y = y plus x)
    ∀ x · (x plus zero = x)
    ...
    ∀ x · (x mult zero = zero)
    ...
END
```

CentraleSupélec
université PARIS-SACLAY

LES TYPES INDUCTIFS

Un type inductif est défini par :

- un type abstrait
- un ou plusieurs constructeurs
- des axiomes d'induction et de disjonction
- éventuellement des fonctions définies par récursion

```
THEORY List
TYPE PARAMETERS
  T
DATATYPES
  List(T) ≙
    ▷ nil
    ▷ cons(head ∈ T, tail ∈ List(T))
OPERATORS
  append(l ∈ List(T), x ∈ T) PREFIX ≙
    recursive definition
      append(nil, x) ≙ cons(x, nil)
      append(cons(y, k), x) ≙ cons(y, append(k, x))
  listsize(l ∈ List(T)) PREFIX ≙
    recursive definition
      listsize(nil) ≙ 0
      listsize(cons(x, k)) ≙ 1 + listsize(k)
  ...
THEOREMS
  @thm1: ∀ l,x · (l ∈ List(T) ∧ x ∈ T
                ⇒ listSize(append(l,x)) = 1 + listSize(l))
  ...
END
```

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

RÔLE DES RÈGLES DE PREUVE DANS UNE THÉORIE

Dans **Event-B**, la majorité des obligations de preuve sont traitées par :

- des prouveurs automatiques
- des règles mathématiques connues

Le plugin **THEORY** permet d'ajouter :

- des règles de réécriture $\longrightarrow$ remplacer une expression (simplification)
- des règles d'inférence $\longrightarrow$ déduire un fait (raisonnement logique)

CentraleSupélec
université PARIS-SACLAY

RÈGLES DE RÉÉCRITURE (REWRITE RULES)

Une **règle de réécriture** permet de remplacer automatiquement une expression par une autre **équivalente**

```
THEORY REAL
AXIOMATIC DEFINITIONS
  TYPES
    ℝ
PROOF RULES
  add_com
    Metavariables
      a ∈ ℝ
      b ∈ ℝ
    Rewrite Rules
      rew1 : a plus b
      rhs1 : T ⊢ b plus a
  ...
END
```

```
THEORY REAL
AXIOMATIC DEFINITIONS
  TYPES
    ℝ
PROOF RULES
  ...
  div_one
    Metavariables
      x ∈ ℝ
    Rewrite Rules
      rew1 :  x div x
      rhs1 : x ≠ zero ⊢ one
  ...
END
```

CentraleSupélec université PARIS-SACLAY

RÈGLES D'INFÉRENCE (INFERENCE RULES)

Une **règle d'inférence** permet de déduire une conclusion à partir de prémisses

$$P_1, P_2, \ldots, P_n \quad \vdash \quad C$$

```
THEORY REAL
AXIOMATIC DEFINITIONS
  TYPES
    ℝ
PROOF RULES
  lt_transitivity
    Metavariables
      a ∈ ℝ
      b ∈ ℝ
      c ∈ ℝ
    Inference Rules
      a lt b (in hypothesis)
      b lt c (in hypothesis)
      --------
      a lt c
  ...
END
```

```
THEORY REAL
AXIOMATIC DEFINITIONS
  TYPES
    ℝ
PROOF RULES
  ...
  leq_antisymetry
    Metavariables
      a ∈ ℝ
      b ∈ ℝ
    Inference Rules
      a leq b (in hypothesis)
      b leq a (in hypothesis)
      --------
      a = b
  ...
END
```

CentraleSupélec
université PARIS-SACLAY

PLAN

- Rappels sur Event-B et Rodin
- Introduction au plugin THEORY
- Définition des types et des opérateurs
- Règles de réécriture et d'inférence
- Conclusion

Retour au plan - Retour à l'accueil

CentraleSupélec
université PARIS-SACLAY

APPORT FONDAMENTAL DU PLUGIN THEORY

Le plugin **THEORY** permet de :

- définir des types abstraits et inductifs
- introduire des opérateurs spécialisés
- formaliser la sémantique par des axiomes et des théorèmes
- guider les prouveurs grâce à des règles de réécriture et d'inférence

CentraleSupélec
université PARIS-SACLAY

MERCI

Version PDF des slides

Retour à l'accueil - Retour au plan

CentraleSupélec
université PARIS-SACLAY